

RUNBOOK NGÀY 3 — SPRINT 1:

AUTHENTICATION & PHÂN QUYỀN

- **Mục tiêu:** Cấu hình ASP.NET Core Identity trên PostgreSQL, xây dựng cơ chế đăng nhập/đăng xuất Cookie cho Blazor Server, seed dữ liệu mẫu (4 roles, 1 admin, 4 demo users), thiết lập bảng user_profiles và teams kèm ràng buộc khóa ngoại (Foreign Keys).
- **Thời lượng dự kiến:** 120 phút
- **Output bắt buộc (DoD):**
 1. Login / Logout hoạt động mượt mà, lưu Session Cookie đúng chuẩn.
 2. Bảng phân quyền: Seed thành công 4 roles (ADMIN, MANAGER, SALES, ACCOUNTANT).
 3. Tài khoản quản trị khởi tạo tự động: admin@crm.local.
 4. Cơ sở dữ liệu: Tạo thành công 7 bảng Identity + 2 bảng nghiệp vụ nhân sự (user_profiles, teams).
 5. Thiết lập hoàn chỉnh 5 ràng buộc FK từ bảng nghiệp vụ sangAspNetUsers.
- **Tech Stack:** .NET 10 SDK, Blazor Server (Interactive Server Render Mode), MudBlazor, PostgreSQL 16, ASP.NET Core Identity 10.0.0, Entity Framework Core.

0. BẢNG TỔNG HỢP CÁC SỬA ĐỔI KỸ THUẬT (CHANGELOG & ADR)

Mã	Vấn đề bản cũ	Giải pháp chuẩn hóa trong Runbook này
S1	AppDbContext tham chiếu entity chưa scaffold (gây lỗi compile).	Thu hẹp phạm vi Ngày 3: chỉ khai báo UserProfile và Team. Các thực thể khác chú thích lại chờ Ngày 4.
S2	Lệch kiểu dữ liệu: Cột tham chiếu user là VARCHAR(450) trong khi Identity dùng UUID.	Đồng bộ toàn bộ *_user_id sang kiểu UUID.
S3	Blazor Server chạy qua WebSocket (SignalR) không thể can thiệp HTTP response header để set Cookie.	Tách rời luồng Auth: Dùng AccountController (HTTP POST) phối hợp JS Interop để set Secure/Lax Cookie, sau đó kích hoạt force reload.

S4	Lỗi PostgreSQL Case-Sensitivity (Nghiêm trọng): Bảng Identity sinh ra cột "Id" (chữ hoa), script SQL cũ tham chiếu id (chữ thường) gây lỗi column "id" does not exist.	Đặt chính xác dấu ngoặc kép "AspNetUsers"("Id") cho tất cả câu lệnh DDL.
M1	Lỗi DI scope lặp khi gọi DbSeeder.	Chuẩn hóa truyền IServiceProvider và resolve dependencies tập trung.
M2	Thiếu package ApexCharts trong Phase 1 nhưng lại gọi trong Program.cs.	Đưa package vào Phase 1 hoặc loại bỏ gọi sớm nếu chưa render biểu đồ.

1. SPECIFICATION & METADATA

Hạng mục	Chi tiết kỹ thuật
Target Framework	.NET 10.0 (LTS)
Database Engine	PostgreSQL 16+ (Npgsql EF Core Provider)
Identity Primary Key	Guid (Map thành UUID trong PostgreSQL)
Password Hasher	PBKDF2 with HMAC-SHA512 (ASP.NET Core default)
Session Lifetime	8 giờ (mặc định), 30 ngày nếu tick chọn "Remember Me"
Lockout Policy	Khóa 15 phút sau 5 lần nhập sai liên tiếp
Schema Tổng hợp	25 bảng (16 bảng nghiệp vụ Ngày 2 + 7 bảng Identity + 2 bảng quản lý nhân sự)

2. CHUẨN BỊ SCHEMA: ĐỒNG BỘ KIỂU DỮ LIỆU USER ID SANG UUID

Do các bảng nghiệp vụ ở Ngày 2 sử dụng kiểu VARCHAR(450) cho các trường tham chiếu người

dùng, cần chuyển đổi sang UUID trước khi tạo bảng Identity.

Cách 1: Reset Database hoàn toàn (Khuyến nghị cho Sprint 1)

Nếu database chưa có dữ liệu quan trọng, xóa và tạo lại database để đảm bảo schema tinh gọn nhất:

```
psql -U postgres -c "DROP DATABASE IF EXISTS crm_db;"
psql -U postgres -c "CREATE DATABASE crm_db WITH ENCODING = 'UTF8' TEMPLATE = template0;"
psql -U postgres -c "GRANT ALL PRIVILEGES ON DATABASE crm_db TO crm_user;"
psql -U crm_user -d crm_db -c "GRANT ALL ON SCHEMA public TO crm_user;"
psql -U crm_user -d crm_db -c "ALTER SCHEMA public OWNER TO crm_user;"
```

Sau đó chạy lại file docs/db/schema.sql của Ngày 2 (lưu ý: đảm bảo các cột *_user_id đã được khai báo là UUID).

Cách 2: Chạy Migration Alter Type (Nếu muốn giữ cấu trúc cũ)

Lưu file sau vào docs/db/01a_fix_user_id_types.sql và thực thi:

```
-- File: docs/db/01a_fix_user_id_types.sql
BEGIN;
```

```
ALTER TABLE customers
  ALTER COLUMN assigned_to_user_id TYPE UUID USING NULL;
```

```
ALTER TABLE customer_assignments
  ALTER COLUMN from_user_id TYPE UUID USING NULL,
  ALTER COLUMN to_user_id TYPE UUID USING NULL,
  ALTER COLUMN assigned_by TYPE UUID USING NULL;
```

```
ALTER TABLE opportunities
  ALTER COLUMN assigned_to_user_id TYPE UUID USING NULL;
```

```
ALTER TABLE stage_histories
  ALTER COLUMN changed_by_user_id TYPE UUID USING NULL;
```

```
ALTER TABLE quotes
  ALTER COLUMN created_by_user_id TYPE UUID USING NULL,
  ALTER COLUMN approved_by TYPE UUID USING NULL;
```

```
ALTER TABLE sales_tasks
  ALTER COLUMN assigned_to_user_id TYPE UUID USING NULL,
```

```
ALTER COLUMN completed_by    TYPE UUID USING NULL;

ALTER TABLE interactions
  ALTER COLUMN user_id TYPE UUID USING NULL;

ALTER TABLE notifications
  ALTER COLUMN user_id TYPE UUID USING NULL;

ALTER TABLE audit_logs
  ALTER COLUMN user_id TYPE UUID USING NULL;

COMMIT;
```

Thực thi:

```
psql -U crm_user -d crm_db -f docs/db/01a_fix_user_id_types.sql
```

PHASE 1: CÀI ĐẶT PACKAGE & CẤU HÌNH DEPENDENCIES (10 phút)

Di chuyển vào thư mục dự án và kiểm tra/cài đặt các thư viện cần thiết:

```
cd D:\Projects\CrmSolution\src\Crm.Web
```

```
# Cài đặt EF Core Identity
```

```
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore --version 10.0.0
```

```
# Đảm bảo các gói UI và Data đã đủ
```

```
dotnet add package MudBlazor --version 8.*
```

```
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL --version 10.*
```

```
cd ../..
```

```
dotnet restore
```

```
dotnet build
```

PHASE 2: TẠO DOMAIN ENTITIES & CẤU HÌNH DBCONTEXT (20 phút)

2.1 Thực thể ApplicationUser

Tạo file src/Crm.Domain/Entities/ApplicationUser.cs:

```
using Microsoft.AspNetCore.Identity;

namespace Crm.Domain.Entities;

/// <summary>
/// Thực thể User định danh, sử dụng khóa chính GUID/UUID.
/// Dữ liệu nghiệp vụ chi tiết được quản lý tại UserProfile.
/// </summary>
public class ApplicationUser : IdentityUser<Guid>
{
    public ApplicationUser()
    {
        Id = Guid.NewGuid();
    }
}
```

2.2 Thực thể UserProfile

Tạo file src/Crm.Domain/Entities/UserProfile.cs:

```
namespace Crm.Domain.Entities;

public class UserProfile
{
    public Guid UserId { get; set; }
    public string FullName { get; set; } = string.Empty;
    public string? EmployeeCode { get; set; }
    public string? Phone { get; set; }
    public string? AvatarUrl { get; set; }
    public string RoleCode { get; set; } = "SALES";
    public int? TeamId { get; set; }
    public decimal MonthlyTarget { get; set; } = 0;
    public bool IsActive { get; set; } = true;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;

    // Navigation properties
    public virtual ApplicationUser? User { get; set; }
    public virtual Team? Team { get; set; }
}
```

2.3 Thực thể Team

Tạo file src/Crm.Domain/Entities/Team.cs:

```
namespace Crm.Domain.Entities;

public class Team
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public Guid? ManagerId { get; set; }
    public bool IsActive { get; set; } = true;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    // Navigation properties
    public virtual ApplicationUser? Manager { get; set; }
    public virtual ICollection<UserProfile> Members { get; set; } = new List<UserProfile>();
}
```

2.4 Cập nhật ApplicationDbContext

Chỉnh sửa file src/Crm.Data/AppDbContext.cs:

```
using Crm.Domain.Entities;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace Crm.Data;

public class ApplicationDbContext : IdentityDbContext<ApplicationUser, IdentityRole<Guid>, Guid>
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options)
    {
    }

    public DbSet<UserProfile> UserProfiles => Set<UserProfile>();
    public DbSet<Team> Teams => Set<Team>();

    // Ghi chú: Các DbSet nghiệp vụ khác (Customers, Deals, Tasks,...)
    // sẽ được kích hoạt tại Sprint 1 - Ngày 4 sau khi scaffold hoàn tất.
```

```

protected override void OnModelCreating(ModelBuilder builder)
{
    base.OnModelCreating(builder);

    // Map UserProfile
    builder.Entity<UserProfile>(entity =>
    {
        entity.ToTable("user_profiles");
        entity.HasKey(e => e.UserId);

        entity.Property(e => e.FullName)
            .IsRequired()
            .HasMaxLength(100);

        entity.Property(e => e.EmployeeCode)
            .HasMaxLength(20);

        entity.Property(e => e.Phone)
            .HasMaxLength(20);

        entity.Property(e => e.AvatarUrl)
            .HasMaxLength(500);

        entity.Property(e => e.RoleCode)
            .IsRequired()
            .HasMaxLength(20);

        entity.Property(e => e.MonthlyTarget)
            .HasColumnType("numeric(18,2)");

        entity.HasOne(e => e.User)
            .WithOne()
            .HasForeignKey<UserProfile>(e => e.UserId)
            .OnDelete(DeleteBehavior.Cascade);

        entity.HasOne(e => e.Team)
            .WithMany(t => t.Members)
            .HasForeignKey(e => e.TeamId)
            .OnDelete(DeleteBehavior.SetNull);
    });

    // Map Team

```

```

builder.Entity<Team>(entity =>
{
    entity.ToTable("teams");
    entity.HasKey(t => t.Id);

    entity.Property(t => t.Name)
        .IsRequired()
        .HasMaxLength(100);

    entity.HasOne(t => t.Manager)
        .WithMany()
        .HasForeignKey(t => t.ManagerId)
        .OnDelete(DeleteBehavior.SetNull);
});
}
}

```

2.5 Cấu hình Program.cs

Cập nhật file src/Crm.Web/Program.cs:

```

using Crm.Data;
using Crm.Domain.Entities;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using MudBlazor.Services;

// Đảm bảo UTC handling cho Npgsql
AppContext.SetSwitch("Npgsql.EnableLegacyTimestampBehavior", false);

var builder = WebApplication.CreateBuilder(args);

// 1. Cấu hình DbContext & Database Connection
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection")
?? throw new InvalidOperationException("Connection string 'DefaultConnection' not found.");

builder.Services.AddDbContext<AppDbContext>(options =>
{
    options.UseNpgsql(connectionString, npgsqlOptions =>
    {
        npgsqlOptions.EnableRetryOnFailure(maxRetryCount: 3);
        npgsqlOptions.CommandTimeout(60);
    });
});

```

```
});
```

```
// 2. Identity Service Configuration
```

```
builder.Services.AddIdentity<ApplicationUser, IdentityRole<Guid>>(options =>
```

```
{
```

```
    // Password policies (.NET 10 production baseline)
```

```
    options.Password.RequireDigit = true;
```

```
    options.Password.RequireLowercase = true;
```

```
    options.Password.RequireUppercase = true;
```

```
    options.Password.RequireNonAlphanumeric = true;
```

```
    options.Password.RequiredLength = 8;
```

```
    options.Password.RequiredUniqueChars = 4;
```

```
    // Lockout settings
```

```
    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
```

```
    options.Lockout.MaxFailedAccessAttempts = 5;
```

```
    options.Lockout.AllowedForNewUsers = true;
```

```
    // User settings
```

```
    options.User.RequireUniqueEmail = true;
```

```
    options.SignIn.RequireConfirmedEmail = false;
```

```
})
```

```
.AddEntityFrameworkStores<AppDbContext>()
```

```
.AddDefaultTokenProviders();
```

```
// Cookie Settings
```

```
builder.Services.ConfigureApplicationCookie(options =>
```

```
{
```

```
    options.Cookie.Name = "CrmSessionCookie";
```

```
    options.Cookie.HttpOnly = true;
```

```
    options.Cookie.SameSite = SameSiteMode.Lax;
```

```
    options.Cookie.SecurePolicy = CookieSecurePolicy.SameAsRequest;
```

```
    options.ExpireTimeSpan = TimeSpan.FromHours(8);
```

```
    options.SlidingExpiration = true;
```

```
    options.LoginPath = "/login";
```

```
    options.LogoutPath = "/logout";
```

```
    options.AccessDeniedPath = "/access-denied";
```

```
});
```

```
// 3. Blazor UI Services & Authentication State
```

```
builder.Services.AddRazorComponents()
```

```
    .AddInteractiveServerComponents();
```

```

builder.Services.AddCascadingAuthenticationState();
builder.Services.AddMudServices();

// 4. API Controllers (Xử lý Login/Logout HTTP Post)
builder.Services.AddControllers();

var app = builder.Build();

// Pipeline Configuration
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error", createScopeForErrors: true);
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseAuthentication();
app.UseAuthorization();
app.UseAntiforgery();

app.MapControllers();

app.MapRazorComponents<Crm.Web.Components.App>()
    .AddInteractiveServerRenderMode();

// 5. Database Seeding Execution
using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;
    var logger = services.GetRequiredService<ILogger<Program>>();
    try
    {
        var env = services.GetRequiredService<IWebHostEnvironment>();
        await DbSeeder.SeedAsync(services, seedDemoData: env.IsDevelopment());
        logger.LogInformation("Database seeded successfully.");
    }
    catch (Exception ex)
    {
        logger.LogError(ex, "An error occurred during database seeding.");
    }
}

```

```
app.Run();
```

PHASE 3: DATABASE SCRIPTS — IDENTITY & USER PROFILES (15 phút)

Cảnh báo kỹ thuật: PostgreSQL phân biệt chữ hoa/thường khi bảng được đặt trong dấu ngoặc kép. ASP.NET Core Identity mặc định sử dụng tên bảng và cột PascalCase (ví dụ: "AspNetUsers", "Id"). Các ràng buộc FK bắt buộc phải trả đúng "AspNetUsers"("Id").

3.1 Script 7 bảng Identity: docs/db/02_identity_tables.sql

```
-- File: docs/db/02_identity_tables.sql
```

```
-- Tạo các bảng AspNet* cho ASP.NET Core Identity (Primary Key: UUID)
```

```
CREATE TABLE IF NOT EXISTS "AspNetRoles" (  
    "Id"          UUID PRIMARY KEY,  
    "Name"        VARCHAR(256),  
    "NormalizedName" VARCHAR(256),  
    "ConcurrencyStamp" TEXT  
);
```

```
CREATE UNIQUE INDEX IF NOT EXISTS "RoleNameIndex"  
    ON "AspNetRoles" ("NormalizedName")  
    WHERE "NormalizedName" IS NOT NULL;
```

```
CREATE TABLE IF NOT EXISTS "AspNetUsers" (  
    "Id"          UUID PRIMARY KEY,  
    "UserName"     VARCHAR(256),  
    "NormalizedUserName" VARCHAR(256),  
    "Email"        VARCHAR(256),  
    "NormalizedEmail" VARCHAR(256),  
    "EmailConfirmed" BOOLEAN NOT NULL DEFAULT FALSE,  
    "PasswordHash" TEXT,  
    "SecurityStamp" TEXT,  
    "ConcurrencyStamp" TEXT,  
    "PhoneNumber" TEXT,  
    "PhoneNumberConfirmed" BOOLEAN NOT NULL DEFAULT FALSE,  
    "TwoFactorEnabled" BOOLEAN NOT NULL DEFAULT FALSE,  
    "LockoutEnd"    TIMESTAMPTZ,  
    "LockoutEnabled" BOOLEAN NOT NULL DEFAULT FALSE,  
    "AccessFailedCount" INT NOT NULL DEFAULT 0
```

);

```
CREATE UNIQUE INDEX IF NOT EXISTS "UserNameIndex"
  ON "AspNetUsers" ("NormalizedUserName")
  WHERE "NormalizedUserName" IS NOT NULL;
```

```
CREATE INDEX IF NOT EXISTS "EmailIndex"
  ON "AspNetUsers" ("NormalizedEmail");
```

```
CREATE TABLE IF NOT EXISTS "AspNetUserRoles" (
  "UserId" UUID NOT NULL REFERENCES "AspNetUsers"("Id") ON DELETE CASCADE,
  "RoleId" UUID NOT NULL REFERENCES "AspNetRoles"("Id") ON DELETE CASCADE,
  PRIMARY KEY ("UserId", "RoleId")
);
```

```
CREATE TABLE IF NOT EXISTS "AspNetUserClaims" (
  "Id" SERIAL PRIMARY KEY,
  "UserId" UUID NOT NULL REFERENCES "AspNetUsers"("Id") ON DELETE CASCADE,
  "ClaimType" TEXT,
  "ClaimValue" TEXT
);
```

```
CREATE INDEX IF NOT EXISTS "IX_AspNetUserClaims_UserId" ON "AspNetUserClaims"
("UserId");
```

```
CREATE TABLE IF NOT EXISTS "AspNetRoleClaims" (
  "Id" SERIAL PRIMARY KEY,
  "RoleId" UUID NOT NULL REFERENCES "AspNetRoles"("Id") ON DELETE CASCADE,
  "ClaimType" TEXT,
  "ClaimValue" TEXT
);
```

```
CREATE INDEX IF NOT EXISTS "IX_AspNetRoleClaims_RoleId" ON "AspNetRoleClaims" ("RoleId");
```

```
CREATE TABLE IF NOT EXISTS "AspNetUserLogins" (
  "LoginProvider" VARCHAR(128) NOT NULL,
  "ProviderKey" VARCHAR(128) NOT NULL,
  "ProviderDisplayName" TEXT,
  "UserId" UUID NOT NULL REFERENCES "AspNetUsers"("Id") ON DELETE CASCADE,
  PRIMARY KEY ("LoginProvider", "ProviderKey")
);
```

```
CREATE INDEX IF NOT EXISTS "IX_AspNetUserLogins_UserId" ON "AspNetUserLogins"
("UserId");
```

```
CREATE TABLE IF NOT EXISTS "AspNetUserTokens" (
```

```

"UserId"    UUID NOT NULL REFERENCES "AspNetUsers"("Id") ON DELETE CASCADE,
"LoginProvider" VARCHAR(128) NOT NULL,
"Name"      VARCHAR(128) NOT NULL,
"Value"     TEXT,
PRIMARY KEY ("UserId", "LoginProvider", "Name")
);

```

3.2 Script User Profiles, Teams & FKs: docs/db/03_user_profiles.sql

```
-- File: docs/db/03_user_profiles.sql
```

```

-- Đảm bảo function trigger tồn tại
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

-- 1. Bảng teams
CREATE TABLE IF NOT EXISTS teams (
    id        SERIAL PRIMARY KEY,
    name      VARCHAR(100) NOT NULL,
    manager_id UUID REFERENCES "AspNetUsers"("Id") ON DELETE SET NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```
CREATE INDEX IF NOT EXISTS idx_teams_manager ON teams(manager_id);
```

```

-- 2. Bảng user_profiles
CREATE TABLE IF NOT EXISTS user_profiles (
    user_id    UUID PRIMARY KEY REFERENCES "AspNetUsers"("Id") ON DELETE CASCADE,
    full_name  VARCHAR(100) NOT NULL,
    employee_code VARCHAR(20) UNIQUE,
    phone      VARCHAR(20),
    avatar_url VARCHAR(500),
    role_code  VARCHAR(20) NOT NULL DEFAULT 'SALES',
    team_id    INT REFERENCES teams(id) ON DELETE SET NULL,
    monthly_target NUMERIC(18,2) DEFAULT 0,
    is_active  BOOLEAN DEFAULT TRUE,

```

```

    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),
    CONSTRAINT chk_role_code CHECK (role_code IN ('ADMIN', 'MANAGER', 'SALES',
'ACCOUNTANT'))
);

```

```

CREATE INDEX IF NOT EXISTS idx_user_profiles_role ON user_profiles(role_code);
CREATE INDEX IF NOT EXISTS idx_user_profiles_team ON user_profiles(team_id);
CREATE INDEX IF NOT EXISTS idx_user_profiles_active ON user_profiles(is_active) WHERE
is_active = TRUE;

```

```

DROP TRIGGER IF EXISTS trg_user_profiles_updated_at ON user_profiles;
CREATE TRIGGER trg_user_profiles_updated_at
    BEFORE UPDATE ON user_profiles
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

```

-- 3. Tạo Foreign Keys tới bảng nghiệp vụ (Nếu bảng tồn tại)

DO \$\$

BEGIN

-- customers

IF EXISTS (SELECT 1 FROM information_schema.tables WHERE table_name = 'customers')
THEN

IF NOT EXISTS (SELECT 1 FROM information_schema.table_constraints WHERE
constraint_name = 'fk_customers_user') THEN

ALTER TABLE customers

ADD CONSTRAINT fk_customers_user

FOREIGN KEY (assigned_to_user_id) REFERENCES "AspNetUsers"("Id")

ON DELETE SET NULL;

END IF;

END IF;

-- opportunities

IF EXISTS (SELECT 1 FROM information_schema.tables WHERE table_name = 'opportunities')
THEN

IF NOT EXISTS (SELECT 1 FROM information_schema.table_constraints WHERE
constraint_name = 'fk_opp_user') THEN

ALTER TABLE opportunities

ADD CONSTRAINT fk_opp_user

FOREIGN KEY (assigned_to_user_id) REFERENCES "AspNetUsers"("Id")

ON DELETE SET NULL;

END IF;

END IF;

```

-- sales_tasks
IF EXISTS (SELECT 1 FROM information_schema.tables WHERE table_name = 'sales_tasks')
THEN
    IF NOT EXISTS (SELECT 1 FROM information_schema.table_constraints WHERE
constraint_name = 'fk_tasks_user') THEN
        ALTER TABLE sales_tasks
            ADD CONSTRAINT fk_tasks_user
            FOREIGN KEY (assigned_to_user_id) REFERENCES "AspNetUsers"("Id")
            ON DELETE RESTRICT;
    END IF;
END IF;

-- interactions
IF EXISTS (SELECT 1 FROM information_schema.tables WHERE table_name = 'interactions')
THEN
    IF NOT EXISTS (SELECT 1 FROM information_schema.table_constraints WHERE
constraint_name = 'fk_interactions_user') THEN
        ALTER TABLE interactions
            ADD CONSTRAINT fk_interactions_user
            FOREIGN KEY (user_id) REFERENCES "AspNetUsers"("Id")
            ON DELETE RESTRICT;
    END IF;
END IF;

-- quotes
IF EXISTS (SELECT 1 FROM information_schema.tables WHERE table_name = 'quotes') THEN
    IF NOT EXISTS (SELECT 1 FROM information_schema.table_constraints WHERE
constraint_name = 'fk_quotes_user') THEN
        ALTER TABLE quotes
            ADD CONSTRAINT fk_quotes_user
            FOREIGN KEY (created_by_user_id) REFERENCES "AspNetUsers"("Id")
            ON DELETE RESTRICT;
    END IF;
END IF;
END $$;

```

3.3 Chạy Migration SQL

```

psql -U crm_user -d crm_db -f docs/db/02_identity_tables.sql
psql -U crm_user -d crm_db -f docs/db/03_user_profiles.sql

```

Kiểm tra số lượng bảng (kỳ vọng: 25 bảng):

```
psql -U crm_user -d crm_db -c "SELECT count(*) FROM information_schema.tables WHERE  
table_schema = 'public';"
```

PHASE 4: DBSEEDER CHO ROLES & USERS (25 phút)

Tạo file src/Crm.Data/DbSeeder.cs với cơ chế kiểm soát Transaction và Error Handling toàn diện:

```
using Crm.Domain.Entities;  
using Microsoft.AspNetCore.Identity;  
using Microsoft.EntityFrameworkCore;  
using Microsoft.Extensions.DependencyInjection;  
using Microsoft.Extensions.Logging;  
  
namespace Crm.Data;  
  
public static class DbSeeder  
{  
    private static readonly string[] Roles = ["ADMIN", "MANAGER", "SALES", "ACCOUNTANT"];  
  
    public static async Task SeedAsync(IServiceProvider serviceProvider, bool seedDemoData =  
false)  
    {  
        var userManager =  
serviceProvider.GetRequiredService<UserManager<ApplicationUser>>();  
        var roleManager =  
serviceProvider.GetRequiredService<RoleManager<IdentityRole<Guid>>>();  
        var dbContext = serviceProvider.GetRequiredService<AppDbContext>();  
        var logger = serviceProvider.GetRequiredService<ILogger<AppDbContext>>();  
  
        // 1. Seed Roles  
        foreach (var roleName in Roles)  
        {  
            if (!await roleManager.RoleExistsAsync(roleName))  
            {  
                var roleResult = await roleManager.CreateAsync(new IdentityRole<Guid>  
                {  
                    Id = Guid.NewGuid(),  
                    Name = roleName,  
                    NormalizedName = roleName.ToUpperInvariant()  
                });  
            }  
        }  
    }  
};
```

```

        if (roleResult.Succeeded)
        {
            logger.LogInformation("Role created: {RoleName}", roleName);
        }
    }
}

// 2. Seed Default Admin
const string adminEmail = "admin@crm.local";
const string adminPassword = "Admin@2026";

var adminUser = await userManager.FindByEmailAsync(adminEmail);
if (adminUser == null)
{
    adminUser = new ApplicationUser
    {
        Id = Guid.NewGuid(),
        UserName = adminEmail,
        Email = adminEmail,
        EmailConfirmed = true,
        LockoutEnabled = true
    };

    var result = await userManager.CreateAsync(adminUser, adminPassword);
    if (result.Succeeded)
    {
        await userManager.AddToRoleAsync(adminUser, "ADMIN");

        var adminProfile = new UserProfile
        {
            UserId = adminUser.Id,
            FullName = "Hệ Thống Quản Trị",
            EmployeeCode = "ADM001",
            RoleCode = "ADMIN",
            IsActive = true
        };

        dbContext.UserProfiles.Add(adminProfile);
        await dbContext.SaveChangesAsync();
        logger.LogInformation("Admin account created: {Email}", adminEmail);
    }
    else

```

```

        {
            logger.LogError("Failed to create Admin: {Errors}", string.Join(", ", result.Errors.Select(e
=> e.Description)));
        }
    }

    // 3. Seed Demo Users & Teams (chỉ chạy trong môi trường Dev)
    if (seedDemoData)
    {
        await SeedDemoDataAsync(userManager, dbContext, logger);
    }
}

private static async Task SeedDemoDataAsync(
    UserManager<ApplicationUser> userManager,
    AppDbContext dbContext,
    ILogger logger)
{
    // Tạo Team mẫu nếu chưa có
    var salesTeam = await dbContext.Teams.FirstOrDefaultAsync(t => t.Name == "Đội Kinh
Doanh 01");
    if (salesTeam == null)
    {
        salesTeam = new Team { Name = "Đội Kinh Doanh 01", IsActive = true };
        dbContext.Teams.Add(salesTeam);
        await dbContext.SaveChangesAsync();
    }

    var demoUsers = new[]
    {
        new { Email = "sales1@crm.local", Password = "Sales@2026", Name = "Nguyễn Văn An",
Code = "SAL001", Role = "SALES", Target = 150000000m },
        new { Email = "sales2@crm.local", Password = "Sales@2026", Name = "Trần Thị Bình", Code
= "SAL002", Role = "SALES", Target = 120000000m },
        new { Email = "manager@crm.local", Password = "Manager@2026", Name = "Lê Hoàng
Nam", Code = "MGR001", Role = "MANAGER", Target = 500000000m },
        new { Email = "accountant@crm.local", Password = "Acc@2026", Name = "Phạm Thu Cúc",
Code = "ACC001", Role = "ACCOUNTANT", Target = 0m }
    };

    foreach (var item in demoUsers)
    {
        if (await userManager.FindByEmailAsync(item.Email) != null) continue;
    }
}

```

```

var user = new ApplicationUser
{
    Id = Guid.NewGuid(),
    UserName = item.Email,
    Email = item.Email,
    EmailConfirmed = true,
    LockoutEnabled = true
};

var createRes = await userManager.CreateAsync(user, item.Password);
if (createRes.Succeeded)
{
    await userManager.AddToRoleAsync(user, item.Role);

    var profile = new UserProfile
    {
        UserId = user.Id,
        FullName = item.Name,
        EmployeeCode = item.Code,
        RoleCode = item.Role,
        MonthlyTarget = item.Target,
        TeamId = item.Role == "SALES" ? salesTeam.Id : null,
        IsActive = true
    };

    dbContext.UserProfiles.Add(profile);

    // Cập nhật manager cho team
    if (item.Role == "MANAGER" && salesTeam.ManagerId == null)
    {
        salesTeam.ManagerId = user.Id;
    }

    logger.LogInformation("Demo user created: {Email} ({Role})", item.Email, item.Role);
}
}

await dbContext.SaveChangesAsync();
}
}

```

PHASE 5: BLUR/COOKIE AUTH PIPELINE & RAZOR UI (40 phút)

5.1 Xây dựng AccountController

Tạo file src/Crm.Web/Controllers/AccountController.cs:

```
using Crm.Domain.Entities;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;

namespace Crm.Web.Controllers;

[Route("api/account")]
[ApiController]
public class AccountController : ControllerBase
{
    private readonly SignInManager<ApplicationUser> _signInManager;
    private readonly ILogger<AccountController> _logger;

    public AccountController(
        SignInManager<ApplicationUser> signInManager,
        ILogger<AccountController> logger)
    {
        _signInManager = signInManager;
        _logger = logger;
    }

    [HttpPost("login")]
    [IgnoreAntiforgeryToken] // Sử dụng trong phạm vi form API từ JS Interop
    public async Task<ActionResult> Login([FromForm] LoginDto request)
    {
        if (!ModelState.IsValid)
        {
            return BadRequest(new { success = false, error = "Dữ liệu đăng nhập không hợp lệ." });
        }

        var result = await _signInManager.PasswordSignInAsync(
            request.Email,
            request.Password,
            request.RememberMe,
            lockoutOnFailure: true);
    }
}
```

```

        if (result.Succeeded)
        {
            _logger.LogInformation("User {Email} logged in successfully.", request.Email);
            return Ok(new { success = true, redirect = string.IsNullOrEmpty(request.ReturnUrl)
? "/" : request.ReturnUrl });
        }

        if (result.IsLockedOut)
        {
            _logger.LogWarning("User {Email} account locked out.", request.Email);
            return BadRequest(new { success = false, error = "Tài khoản tạm thời bị khóa 15 phút do
nhập sai 5 lần liên tiếp." });
        }

        if (result.IsNotAllowed)
        {
            return BadRequest(new { success = false, error = "Tài khoản chưa được xác thực hoặc bị
vô hiệu hóa." });
        }

        return BadRequest(new { success = false, error = "Email hoặc mật khẩu không chính xác." });
    }

    [HttpPost("logout")]
    [Authorize]
    [IgnoreAntiforgeryToken]
    public async Task<ActionResult> Logout()
    {
        await _signInManager.SignOutAsync();
        return Ok(new { success = true });
    }
}

public class LoginDto
{
    public string Email { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
    public bool RememberMe { get; set; }
    public string? ReturnUrl { get; set; }
}

```

5.2 Xây dựng JS Interop

Tạo file src/Crm.Web/wwwroot/js/auth-interop.js:

```
window.crmAuth = {
  login: async function (email, password, rememberMe, returnUrl) {
    const formData = new FormData();
    formData.append('Email', email);
    formData.append('Password', password);
    formData.append('RememberMe', rememberMe);
    formData.append('ReturnUrl', returnUrl || '/');

    try {
      const response = await fetch('/api/account/login', {
        method: 'POST',
        body: formData,
        credentials: 'include'
      });

      const data = await response.json();
      if (response.ok && data.success) {
        return { success: true, redirect: data.redirect, error: null };
      }
      return { success: false, redirect: null, error: data.error || 'Đăng nhập không thành công.' };
    } catch (err) {
      console.error('Login network error:', err);
      return { success: false, redirect: null, error: 'Không thể kết nối đến máy chủ xác thực.' };
    }
  },

  logout: async function () {
    try {
      await fetch('/api/account/logout', {
        method: 'POST',
        credentials: 'include'
      });
    } catch (err) {
      console.error('Logout error:', err);
    } finally {
      window.location.href = '/login';
    }
  }
};
```

5.3 EmptyLayout & Shared Components

File: src/Crm.Web/Components/Layout/EmptyLayout.razor

@inherits LayoutComponentBase

<MudThemeProvider />

<MudDialogProvider />

<MudSnackbarProvider />

<MudLayout>

 <MudMainContent Class="pa-0">

 @Body

 </MudMainContent>

</MudLayout>

File: src/Crm.Web/Components/Shared/RedirectToLogin.razor

@inject NavigationManager Navigation

@code {

 protected override void OnInitialized()

 {

 var currentRelativeUri = Navigation.ToBaseRelativePath(Navigation.Uri);

 var target = \$"/login?returnUrl={Uri.EscapeDataString(currentRelativeUri)}";

 Navigation.NavigateTo(target, forceLoad: true);

 }

}

5.4 Trang Đăng nhập Login.razor

Tạo file src/Crm.Web/Components/Pages/Account/Login.razor:

@page "/login"

@layout EmptyLayout

@rendermode InteractiveServer

@inject IJSRuntime JS

@inject NavigationManager Navigation

@inject ILogger<Login> Logger

<PageTitle>Đăng nhập hệ thống — CRM</PageTitle>

<div class="d-flex align-center justify-center" style="min-height: 100vh; background: #f0f2f5;">
 <MudPaper Elevation="4" Class="pa-8" Style="width: 100%; max-width: 440px; border-radius: 12px;">

<div class="d-flex flex-column align-center mb-6">
 <MudIcon Icon="@Icons.Material.Filled.Shield" Color="Color.Primary" Style="font-size: 52px;" />

<MudText Typo="Typo.h5" Class="font-weight-bold mt-2">HỆ THỐNG CRM</MudText>
 <MudText Typo="Typo.body2" Color="Color.Secondary">Vui lòng đăng nhập để tiếp tục</MudText>

</div>

<EditForm Model="@_loginModel" OnValidSubmit="HandleLoginSubmit">
 <DataAnnotationsValidator />

<MudTextField @bind-Value="_loginModel.Email"
 Label="Email công ty"
 Variant="Variant.Outlined"
 InputType="InputType.Email"
 Adornment="Adornment.Start"
 AdornmentIcon="@Icons.Material.Filled.Email"
 For="@(() => _loginModel.Email)"
 Disabled="@_isLoading"
 Class="mb-3" />

<MudTextField @bind-Value="_loginModel.Password"
 Label="Mật khẩu"
 Variant="Variant.Outlined"
 InputType="@_passwordInputType"
 Adornment="Adornment.Start"
 AdornmentIcon="@Icons.Material.Filled.Lock"
 AdornmentEndIcon="@_passwordToggleIcon"
 OnAdornmentClick="TogglePasswordVisibility"
 For="@(() => _loginModel.Password)"
 Disabled="@_isLoading"
 Class="mb-2" />

<div class="d-flex justify-space-between align-center mb-4">
 <MudCheckBox @bind-Value="_loginModel.RememberMe"
 Label="Ghi nhớ 30 ngày"
 Color="Color.Primary"
 Dense="true" />

```

</div>

@if (!string.IsNullOrEmpty(_errorMessage))
{
    <MudAlert Severity="Severity.Error" Dense="true" Class="mb-4">
        @_errorMessage
    </MudAlert>
}

<MudButton ButtonType="ButtonType.Submit"
    Variant="Variant.Filled"
    Color="Color.Primary"
    FullWidth="true"
    Size="Size.Large"
    Disabled="@_isLoading">
    @if (_isLoading)
    {
        <MudProgressCircular Size="Size.Small" Indeterminate="true" Class="mr-2" />
        <span>Đang xác thực...</span>
    }
    else
    {
        <span>Đăng nhập</span>
    }
</MudButton>
</EditForm>
</MudPaper>
</div>

@code {
    [SupplyParameterFromQuery]
    public string? returnUrl { get; set; }

    private readonly LoginViewModel _loginModel = new();
    private bool _isLoading;
    private bool _showPassword;
    private string? _errorMessage;

    private InputType _passwordInputType => _showPassword ? InputType.Text :
    InputType.Password;
    private string _passwordToggleIcon => _showPassword ? Icons.Material.Filled.VisibilityOff :
    Icons.Material.Filled.Visibility;

```

```
private void TogglePasswordVisibility() => _showPassword = !_showPassword;
```

```
private async Task HandleLoginSubmit()
```

```
{
```

```
    _isLoading = true;
```

```
    _errorMessage = null;
```

```
    try
```

```
    {
```

```
        var res = await JS.InvokeAsync<LoginResult>("crmAuth.login",
```

```
            _loginModel.Email,
```

```
            _loginModel.Password,
```

```
            _loginModel.RememberMe,
```

```
            returnUrl ?? "/");
```

```
        if (res.Success)
```

```
        {
```

```
            Navigation.NavigateTo(res.Redirect ?? "/", forceLoad: true);
```

```
        }
```

```
        else
```

```
        {
```

```
            _errorMessage = res.Error;
```

```
        }
```

```
    }
```

```
    catch (Exception ex)
```

```
    {
```

```
        Logger.LogError(ex, "Unexpected error during login execution.");
```

```
        _errorMessage = "Lỗi kết nối hệ thống. Vui lòng thử lại.";
```

```
    }
```

```
    finally
```

```
    {
```

```
        _isLoading = false;
```

```
    }
```

```
}
```

```
private class LoginViewModel
```

```
{
```

```
    [System.ComponentModel.DataAnnotations.Required(ErrorMessage = "Vui lòng nhập Email")]
```

```
    [System.ComponentModel.DataAnnotations.EmailAddress(ErrorMessage = "Email không hợp lệ")]
```

```
    public string Email { get; set; } = string.Empty;
```

```

[System.ComponentModel.DataAnnotations.Required(ErrorMessage = "Vui lòng nhập mật khẩu")]
public string Password { get; set; } = string.Empty;

public bool RememberMe { get; set; }
}

private class LoginResult
{
    public bool Success { get; set; }
    public string? Redirect { get; set; }
    public string? Error { get; set; }
}
}

```

5.5 Trang Đăng xuất Logout.razor & AccessDenied

File: src/Crm.Web/Components/Pages/Account/Logout.razor

```

@page "/logout"
@layout EmptyLayout
@rendermode InteractiveServer
@inject IJSRuntime JS

<PageTitle>Đang đăng xuất...</PageTitle>

<div class="d-flex flex-column align-center justify-center" style="min-height: 100vh;">
    <MudProgressCircular Indeterminate="true" Color="Color.Primary" Size="Size.Large" />
    <MudText Typo="Typo.body1" Class="mt-4">Đang đóng phiên làm việc an toàn...</MudText>
</div>

@code {
    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (firstRender)
        {
            await JS.InvokeVoidAsync("crmAuth.logout");
        }
    }
}

```

File: src/Crm.Web/Components/Pages/Account/AccessDenied.razor

```
@page "/access-denied"
@layout EmptyLayout

<PageTitle>Từ chối truy cập</PageTitle>

<div class="d-flex align-center justify-center" style="min-height: 100vh;">
    <MudPaper Class="pa-8 d-flex flex-column align-center" Elevation="2" Style="max-width:
480px;">
        <MudIcon Icon="@Icons.Material.Filled.GppBad" Color="Color.Error" Style="font-size:
64px;" />
        <MudText Typo="Typo.h5" Class="mt-4">Truy Cập Bị Từ Chối</MudText>
        <MudText Typo="Typo.body2" Color="Color.Secondary" Class="text-center mt-2">
            Tài khoản của bạn không có đủ phân quyền (Role) để xem trang này. Vui lòng liên hệ
Administrator.
        </MudText>
        <MudButton Variant="Variant.Filled" Color="Color.Primary" Href="/" Class="mt-6">
            Quay lại Trang Chủ
        </MudButton>
    </MudPaper>
</div>
```

5.6 Cập nhật Routes.razor, App.razor & _Imports.razor

File: src/Crm.Web/Components/Routes.razor

```
@using Microsoft.AspNetCore.Components.Authorization
@using Crm.Web.Components.Shared
@using Crm.Web.Components.Layout

<Router AppAssembly="@typeof(Program).Assembly">
    <Found Context="routeData">
        <AuthorizeRouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)">
            <NotAuthorized>
                @if (context.User.Identity?.IsAuthenticated != true)
                {
                    <RedirectToLogin />
                }
                else
                {
                    <NavigationManager NavigationManager.NavigateTo("/access-denied") />
                }
            </NotAuthorized>
        </AuthorizeRouteView>
    </Found>
</Router>
```

```

        </NotAuthorized>
    </AuthorizeRouteView>
    <FocusOnNavigate RouteData="@routeData" Selector="h1" />
</Found>
<NotFound>
    <PageTitle>Không tìm thấy trang</PageTitle>
    <LayoutView Layout="@typeof(MainLayout)">
        <MudAlert Severity="Severity.Warning">Đường dẫn không tồn tại.</MudAlert>
    </LayoutView>
</NotFound>
</Router>

```

Nhúng file script tại src/Crm.Web/Components/App.razor:

```

<head>
    <!-- Các thẻ meta và link style của MudBlazor -->
    <link href="_content/MudBlazor/MudBlazor.min.css" rel="stylesheet" />
</head>
<body>
    <Routes @rendermode="InteractiveServer" />

    <script src="_framework/blazor.web.js"></script>
    <script src="_content/MudBlazor/MudBlazor.min.js"></script>
    <script src="js/auth-interop.js"></script>
</body>

```

File: src/Crm.Web/Components/_Imports.razor

Thêm vào cuối file:

```

@using Microsoft.AspNetCore.Authorization
@using Microsoft.AspNetCore.Components.Authorization
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.JSInterop
@using MudBlazor
@using Crm.Domain.Entities
@using Crm.Web.Components.Layout
@using Crm.Web.Components.Shared

```

PHASE 6: E2E VERIFICATION & TEST MATRIX (10 phút)

Khởi động hệ thống:

cd src/Crm.Web

dotnet run

Bảng Kiểm thử 7 Tiêu chí DoD (Definition of Done)

Mã	Kịch bản kiểm thử	Hành động thực hiện	Kết quả đạt chuẩn (Expected)	Trạng thái
TC1	Build & Startup	dotnet run	Ứng dụng chạy trên cổng HTTPS (vd: 5001), console log ghi nhận seed Roles & Admin thành công.	[] PASS
TC2	Unauthenticated Redirect	Mở tab ẩn danh truy cập https://localhost:5001/	Tự động chuyển hướng về /login?returnUrl=%2F	[] PASS
TC3	Form UI & Styling	Xem trang /login	Render đúng giao diện MudBlazor, căn giữa, icon, responsive.	[] PASS
TC4	Sai thông tin đăng nhập	Nhập admin@crm.local / SaiPass123	Báo lỗi: "Email hoặc mật khẩu không chính xác."	[] PASS
TC5	Đăng nhập thành công	Nhập admin@crm.local / Admin@2026	Chuyển hướng vào trang chính /, Cookie CrmSessionCookie được cấp.	[] PASS

TC6	Account Lockout	Cố tình nhập sai mật khẩu 5 lần liên tiếp	Báo lỗi: " <i>Tài khoản tạm thời bị khóa 15 phút do nhập sai 5 lần liên tiếp.</i> "	[] PASS
TC7	Đăng xuất an toàn	Nhấn gọi URL /logout	Cookie bị hủy bỏ, trình duyệt quay lại /login.	[] PASS

Kiểm tra tính toàn vẹn Database

```
psql -U crm_user -d crm_db -c "SELECT count(*) AS total_users FROM \"AspNetUsers\";"
psql -U crm_user -d crm_db -c "SELECT count(*) AS total_roles FROM \"AspNetRoles\";"
psql -U crm_user -d crm_db -c "SELECT user_id, full_name, role_code, monthly_target FROM user_profiles;"
```

Kỳ vọng: Có 5 Users (1 admin + 4 demo), 4 Roles chuẩn, 5 bản ghi User Profiles.

PHASE 7: GIT COMMIT & DAILY LOG (5 phút)

```
cd D:\Projects\CrmSolution
git add .
git commit -m "feat(auth): complete day 3 identity & authentication setup"
```

- Resolved Postgres case-sensitive issues with AspNetUsers quoted identifiers
- Configured ApplicationUser with Guid PK and clean UserProfile separation
- Scaffolded Identity schema and added FK constraints to domain tables
- Implemented MVC Controller + JS Interop Cookie authentication for Blazor Server
- Configured account lockout policy (5 attempts, 15m) and Sliding Session
- Seeded default roles (ADMIN, MANAGER, SALES, ACCOUNTANT) and system accounts
- Verified 7/7 DoD criteria"

Cập nhật file notes/daily.md:

Daily Log — Ngày 3 / Sprint 1

Đã hoàn thành (Done)

- [x] Sửa triệt để kiểu dữ liệu UUID và các khóa ngoại liên kết tới AspNetUsers.
- [x] Khởi tạo thành công 7 bảng Identity + `user\_profiles` + `teams`.
- [x] Cấu hình hoàn tất pipeline Cookie Authentication với Controller & JS Interop.

- [x] Hoàn thiện giao diện Login, Logout, AccessDenied với MudBlazor.
- [x] Seed đầy đủ 4 Roles, 1 Admin và 4 Demo Users.
- [x] Test 7/7 DoD tiêu chí đạt chuẩn.

Danh sách tài khoản thử nghiệm

- ****Admin:**** `admin@crm.local` / `Admin@2026` (Quyền: ADMIN)
- ****Quản lý:**** `manager@crm.local` / `Manager@2026` (Quyền: MANAGER)
- ****Sales 1:**** `sales1@crm.local` / `Sales@2026` (Quyền: SALES)
- ****Sales 2:**** `sales2@crm.local` / `Sales@2026` (Quyền: SALES)
- ****Kế toán:**** `accountant@crm.local` / `Acc@2026` (Quyền: ACCOUNTANT)

Kế hoạch Ngày 4

- Scaffold Entity cho 16 bảng nghiệp vụ còn lại.
- Mở khóa toàn bộ DbSet trong `AppDbContext`.
- Thiết lập Value Converter cho UTC DateTimes.
- Viết Seeder dữ liệu mẫu ban đầu (Khách hàng, Cơ hội, Tương tác).

git add notes/daily.md

git commit -m "docs: record day 3 daily log"

git push origin develop

BẢNG TRA CỨU SỬA LỖI NHANH (TROUBLESHOOTING)

Lỗi gặp phải	Nguyên nhân gốc rễ	Giải pháp xử lý
column "id" does not exist khi tạo FK	Bảng Identity tạo cột "Id" (chữ hoa), script SQL trở vào id (không bọc nháy kép).	Sửa lại câu lệnh: REFERENCES "AspNetUsers"("Id").
Login thành công nhưng reload vẫn bị văng về /login	Cookie chưa được trình duyệt lưu do cấu hình SameSite hoặc thiếu cờ credentials: 'include'.	Kiểm tra auth-interop.js đã có credentials: 'include' và Program.cs cấu hình SameSiteMode.Lax.
Lỗi Antiforgery Token khi POST form login	Blazor Server kích hoạt antiforgery toàn cầu trên .NET 8/10.	Thêm [IgnoreAntiforgeryToken] trên endpoint Login

		Controller vì đã bảo vệ qua CORS/Origin nội bộ.
Tài khoản Admin bị khóa ngoài ý muốn	Test nhập sai mật khẩu quá 5 lần.	Chạy SQL giải phóng: UPDATE "AspNetUsers" SET "LockoutEnd" = NULL WHERE "Email" = 'admin@crm.local';
MudBlazor icon hoặc font không hiển thị	Thiếu các thẻ link CSS/JS của MudBlazor trong App.razor.	Kiểm tra lại mục 5.6, đảm bảo đã nhúng cả MudBlazor.min.css và MudBlazor.min.js.